

Data Analysis with Python 2024–2025

Parte 4: Pandas (DataFrames na Prática) Soluções dos Exercícios

Soluções independentes elaboradas para fins de estudo, com base nos enunciados originais

Nota Importante

Este material é uma adaptação das soluções independentes para os exercícios baseados no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**. As soluções abaixo não são o gabarito oficial do curso.

Conteúdo

1 Soluções - Pandas na Prática

A seguir estão as soluções propostas para os exercícios da lista complementar. Todos os trechos assumem que o `pandas` e `numpy` já foram importados como `pd` e `np`.

Exercício 1 – Capitais

Solução proposta

```
import pandas as pd

def capitais():
    dados = {
        "Capital": ["Recife", "Salvador", "Fortaleza", "Curitiba",
                    "Manaus"],
        "Populacao": [1653461, 2418616, 2428708, 1963726, 2219580]
    }
    indice = ["Pernambuco", "Bahia", "Ceara", "Parana", "Amazonas"]
    return pd.DataFrame(dados, index=indice)
```

Exercício 2 – Tabela de potências

Solução proposta

```
import pandas as pd

def tabela_de_potencias(serie, k):
    colunas = {expoente: serie ** expoente for expoente in range(1,
        k + 1)}
    return pd.DataFrame(colunas, index=serie.index)
```

Aqui construímos um dicionário em que cada chave é um expoente (de 1 até k) e o valor é a série elevada a esse expoente. Como o `pandas` preserva a ordem das chaves do dicionário ao montar as colunas, o resultado já sai na ordem correta.

Exercício 3 – Carregando um arquivo tabulado

Solução proposta

```
import pandas as pd

def carregar_vendas(caminho):
    df = pd.read_csv(caminho, sep="\t")
    linhas, colunas = df.shape
    print(f"Formato: {linhas}, {colunas}")
    print("Colunas:")
    for nome_coluna in df.columns:
        print(nome_coluna)
    return df
```

O parâmetro `sep="\t"` instrui o `read_csv` a interpretar o caractere de tabulação como separador de campos, já que o arquivo não usa vírgulas.

Exercício 4 – Filtro de funcionários

Solução proposta

```
def salarios_altos(df, limite):  
    filtro = df["salario"] > limite  
    return df.loc[filtro, ["nome", "salario"]]
```

Primeiro construímos uma máscara booleana comparando a coluna de salários com o limite; em seguida usamos `loc` para aplicar essa máscara às linhas e, ao mesmo tempo, restringir o resultado às colunas desejadas – tudo em uma única operação.

Exercício 5 – Seleção com `loc`

Solução proposta

```
def selecionar_com_loc(df):  
    return df.loc["Ana":"Diego", ["salario", "setor"]]
```

Ao contrário das fatias comuns do Python, uma fatia de rótulos com `loc` inclui o limite superior – por isso "Diego" aparece no resultado.

Exercício 6 – Seleção com `iloc`

Solução proposta

```
def top5_por_posicao(df):  
    return df.iloc[0:5, 0:2]
```

Com `iloc`, os limites funcionam como fatias comuns do Python (o limite superior é exclusivo), daí selecionarmos até a posição 5 para obter as 5 primeiras linhas.

Exercício 7 – Temperatura máxima

Solução proposta

```
def temperatura_maxima(df):  
    return df["Temperatura (C)"].max()  
  
# No programa principal:  
# maximo = temperatura_maxima(df)  
# print(f"Temperatura maxima: {maximo:.1f}")
```

Exercício 8 – Média de um mês específico

Solução proposta

```
def media_do_mes(df, mes):  
    subset = df[df["mes"] == mes]  
    return subset["Temperatura (C)"].mean()
```

Exercício 9 – Dias de chuva

Solução proposta

```
def dias_de_chuva(df):  
    return (df["Precipitacao (mm)"] > 0).sum()
```

A comparação produz uma série booleana (**True/False**); somar essa série equivale a contar quantos valores são **True**, já que o Python trata **True** como 1 e **False** como 0.

Exercício 10 – Limpeza de um sensor com falhas

Solução proposta

```
import pandas as pd  
  
def limpar_sensor(caminho):  
    df = pd.read_csv(caminho)  
    df = df.dropna(axis=0, how="all") # remove linhas totalmente vazias  
    df = df.dropna(axis=1, how="all") # remove colunas totalmente vazias  
    return df
```

O parâmetro `how="all"` garante que uma linha (ou coluna) só será descartada quando todos os seus valores estiverem ausentes – diferente do comportamento padrão de `dropna`, que remove uma linha assim que encontra qualquer valor faltando.

Exercício 11 – Tipos de valores ausentes

Solução proposta

```
import pandas as pd  
import numpy as np  
  
def tabela_com_lacunhas():  
    dados = {  
        "Fundacao": [1822, 1816, np.nan, 1825, 1811],  
        "Capital": ["Brasilia", "Buenos Aires", "Santiago", None, "Assuncao"]  
    }  
    indice = ["Brasil", "Argentina", "Chile", "Uruguai", "Paraguai"]  
    df = pd.DataFrame(dados, index=indice)  
    df.index.name = "Pais"  
    return df
```

Usamos `np.nan` para a lacuna na coluna numérica (o que automaticamente promove essa coluna para o tipo `float64`) e `None` para a lacuna na coluna de texto (mantendo o tipo `object`).

Exercício 12 – Separando nome completo e normalizando texto

Solução proposta

```
def separar_nomes(serie):  
    partes = serie.str.split(expand=True)  
    partes.columns = ["Primeiro_nome", "Sobrenome"]  
    return partes.apply(lambda coluna: coluna.str.capitalize())
```

Primeiro dividimos cada string em duas colunas com `str.split(expand=True)`. Depois renomeamos as colunas e aplicamos `str.capitalize()` a cada uma delas para deixar a primeira letra maiúscula, mantendo o restante em minúsculas.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.